



Manual de Iniciación a Oracle



ORACLE



Detodoprogramación.com

Créditos

Este manual pertenece a la empresa Expertos en Servicios de Consultoría Exes, S.L., que tiene todos los derechos sobre esta obra.

Asimismo se prohíbe expresamente su reproducción, transformación, fijación, distribución en cualquiera de sus formas (total, parcial, en papel, en formato electrónico) incluyendo el tratamiento informático del documento en cualquier parte del mundo de acuerdo con las Leyes y Convenios Internacionales.

La utilización de este documento por cualquier persona diferente a la compradora del derecho de uso está expresamente prohibida.

El contenido de este manual está protegido por la ley penal vigente, ateniéndose, en caso de mal uso, a responder frente a la justicia.

El fin de este documento es el uso particular del comprador de su uso, quedando expresamente prohibida la utilización por empresas o centros de formación, o cualquier otro, incluyendo a los empleados de cualquier tipo, colaboradores o alumnos.

Si alguna persona está interesada en la utilización de este manual, se puede poner en contacto con Exes en www.exes.es o en exes@exes.es.

Marcas Comerciales: Las designaciones utilizadas por las empresas para distinguir sus productos suelen ser marcas registradas. Exes solo ha utilizado estas marcas en beneficio de sus fabricantes, dado el carácter divulgativo de la formación. Los ejemplos y pantallas no se consideran reales.

Se ha puesto el máximo empeño en ofrecer al usuario una información completa del tema tratado. Sin embargo, Exes no asume ninguna responsabilidad derivada de su uso, ni de la violación de las patentes ni otros derechos de terceras partes que pudieran ocurrir.

TEMA 1 EL MODELO RELACIONAL

El modelo relacional

- E.F. Codd propuso el modelo relacional para sistemas de b.d. En 1970.
- Sustituyó a los modelos más populares del momento: el jerárquico y el de red. Facilidad de uso y flexibilidad.

- Constituye la base para el sistema manejador de bd. Relacionales (RDBMS).

El modelo de datos E-R

- El modelo de datos E-R se basa en una percepción de un mundo real que consiste en:
 - Una colección de objetos básicos → ENTIDADES
 - Relaciones entre estos objetos → RELACIONES
 - Restricciones de los datos → Cardinalidad de Asignación
- Una entidad se compone de Atributos.

Diagrama E-R

- Mundo real → Modelo conceptual (E-R)
- Estructura de un diagrama E-R:
 - RECTÁNGULOS → Entidades
 - ELIPSES → Atributos
 - ROMBOS → Relaciones
 - LÍNEAS → Conectores
- Ejemplo de un diagrama E-R:

Terminología de una b.d. relacional

Relaciones entre tablas

Oracle 8: ORDBMS

- Posibilidad de almacenar objetos (extensión a Oracle7)
 - Modelo de datos relacional orientado a objeto
 - Ventajas de la programación orientada a objetos
- Respecto a Oracle7:
 - > rendimiento y funcionalidad de transacciones online
 - Soporte de aplicaciones cliente-servidor
 - Soporte de aplicaciones basadas en Web distribuidas
 - Escalabilidad a decenas de miles de usuarios
 - Hasta 512 petabytes
 - Manejo de datos multimedia (imagen sonido, video).

TEMA 2

EL LENGUAJE S.Q.L.

¿Qué es S.Q.L.

- Structured Query Language
- Establecido como el lenguaje de base de datos relacional estándar.
- Existen numerosos productos que soportan SQL, cada uno de ellos con pequeñas diferencias sin apenas importancia (p.ej. Oracle).
- El SQL estándar es el publicado por ANSI e ISO.

Características de S.Q.L.

- Lenguaje de definición de datos (DDL)
 - Create, Alter, Drop.
- Lenguaje de manipulación de datos (DML)
 - Insert, Update, Delete.

- Lenguaje de control de datos (DCL)
 - Grant, Revoke.
- Control de transacciones
 - Commit, Rollback, Savepoint
- Restricciones de integridad
 - Referencial, datos.

S.Q.L.

La sentencia SELECT

S.Q.L.

Sentencia Básica

- Ejemplos:

SELECT	empno,	ename
FROM		emp;
SELECT		*
FROM emp;		

S.Q.L.

Etiquetas de Columnas por defecto

- Justificación de etiquetas y datos por defecto:
 - Izquierda → fechas y caracteres
 - Derecha → datos numéricos
- Etiquetas y datos, por defecto, se muestran en mayúsculas.
- Posibilidad del uso de alias de columnas.

S.Q.L.

Alias de columna

- Un alias de columna renombra un encabezamiento de columna.
- Útil, especialmente, en cálculos.
- Sintaxis:
 - Sigue inmediatamente al nombre de la columna mediante la palabra clave AS entre ellos.
 - Se requiere encerrar un alias entre comillas dobles si contiene espacios en blanco, caracteres especiales o es *case sensitive*.

S.Q.L.

Uso de Alias de columna

- SQL> SELECT ename AS nombre, sal salario, sal*12 AS "Salario Anual" FROM emp;
- Resultado:

S.Q.L.

Expresiones aritméticas

- Operadores:
 - Suma (+)
 - Resta (-)
 - Multiplicación (*)
 - División (/)
- Cualquier sentencia SQL
- Precedencia de operadores: * / + -
- Operadores misma prioridad se evalúan de izq. a derecha.
- Paréntesis sobrescriben reglas de precedencia.

S.Q.L.

Valor NULL

- NULL es un valor inaccesible, sin valor desconocido o inaplicable.
- NULL no representa ni un cero ni un blanco.
- Las expresiones aritméticas que contengan NULL se evalúan a NULL (<> cero).

S.Q.L.

Operador de concatenación

- Representado por dos barras verticales: ||
- Vincula columnas o cadenas de caracteres.
- Crea una columna resultado que es una expresión de tipo carácter.
- Ejemplo:

S.Q.L.

Filas duplicadas

- Las consultas, por defecto, muestran todas las filas, incluyendo las duplicadas.
- Las duplicadas se eliminan usando DISTINCT en la cláusula SELECT:

S.Q.L.

Uso de la cláusula WHERE

- Establece un criterio de selección aplicable a la tabla de la que se quiere recuperar datos.
- Se especifica a continuación de FROM.
- La condición de WHERE permite comparar columnas con otras columnas, valores, literales, expresiones aritméticas o funciones.

S.Q.L.

Sintaxis de WHERE

- Ejemplo:

S.Q.L.

Operadores de comparación

- Aplicables en las condiciones de la cláusula WHERE:
 - = Igual que
 - > Mayor que
 - >= Mayor que o igual a
 - < Menor que
 - <= Menor que o igual a
 - <> Distinto
 - BETWEEN m AND n → Entre m y n (inclusive)
 - IN(lista) → Se encuentra en la lista
 - LIKE → Se ajusta a un patrón
 - IS NULL → Es valor nulo

S.Q.L.

Ejemplos de comparación

- SQL> SELECT ename, sal FROM emp
WHERE sal **BETWEEN** 1000 **AND** 1500;
- SQL> SELECT empno, ename, sal, mgr FROM EMP
WHERE mgr **IN** (7902, 7566, 7788);

- Sintaxis de LIKE:
 - “%” representa cero o varios caracteres
 - “_” representa un solo carácter
- SQL> SELECT ename, mgr FROM emp
WHERE mgr IS NULL;

Operadores lógicos

- S.Q.L.**

- SQL> SELECT empno, ename, job, sal FROM emp
WHERE sal >= 1100
AND job = 'CLERK';
- SQL> SELECT empno, ename, job, sal FROM emp
WHERE sal >= 2000
OR job = 'MANAGER';
- SQL> SELECT ename, job FROM emp
WHERE job **NOT** IN ('CLERK', 'MANAGER', 'ANALYST');

Reglas de precedencia

- www.detodoprogramacion.com

- 2: NOT
- 3: AND
- 4: OR

S.Q.L.

Recuperaciones ordenadas

- Las filas recuperadas en la sentencia SELECT, si no se especifica nada, no tienen ningún orden determinado.
- Se pueden ordenar con la cláusula ORDER BY siempre al final de una sentencia SELECT.
 - ASC → Orden ascendente. Por defecto.
 - DESC → Orden descendente.

S.Q.L.

Sintaxis de ORDER BY

- Ejemplo:

TEMA 3

FUNCIONES S.Q.L.

S.Q.L.

Funciones SQL

- Existen dos tipos de funciones:
 - Funciones a nivel de fila
 - Funciones a nivel de múltiples filas
- Funciones a nivel de fila. Tipos:

- Carácter
 - Número
 - Fecha
 - Conversión
- Funciones a nivel de grupo. Tipos:
 - Manipulan grupos de filas y devuelven un resultado por cada uno de ellos.

S.Q.L.

Funciones de Caracteres

- Pueden ser de dos tipos:
 - Funciones de conversión
 - Funciones de manipulación de caracteres
- De manipulación de caracteres:
 - CONCAT
 - SUBSTR
 - LENGTH
 - INSTR
 - LPAD Y RPAD
- De conversión:
 - LOWER , UPPER e INITCAP

S.Q.L.

Funciones de conversión caracteres

- LOWER: Convierte a minúsculas.
- UPPER: Convierte a mayúsculas.

- INITCAP: Convierte la primera letra de cada palabra en mayúsculas, y el resto en minúscula.
- *Atención:* Usar una función de conversión dentro de la cláusula WHERE puede ser altamente ineficiente porque si la columna afectada forma parte de un índice éste lo desactiva, provocando un bajo rendimiento.

S.Q.L.

Funciones manipulación caracteres

- CONCAT: Concatena dos valores.
- SUBSTR: Extrae una subcadena.
- LENGTH: Devuelve la longitud de la cadena.
- INSTR: Devuelve la posición de un carácter o subcadena.
- LPAD: Justifica a la derecha la cadena.
- RPAD: Justifica a la izquierda la cadena.

S.Q.L.

Funciones Numéricas

- ROUND (columna | expresión, n)
 - Redondea a n posiciones decimales. Si se omite n, no se redondea con decimales. Si n es negativo, los números a la izquierda del punto decimal se redondean a decenas, centenas, ...
- TRUNC (columna | expresión, n)
 - Trunca en la enésima posición decimal. Si se omite n, sin lugares decimales. Si n es negativo, los números a la izquierda del punto decimal se truncan a cero.
- MOD (m, n)
 - Devuelve el resto de la división de m por n.

S.Q.L.

Ejemplos de funciones numéricas

- SQL> SELECT ROUND(45.923, 2), ROUND(45.923, 0), ROUND(45.923, -1)
FROM SYS.DUAL;
- Resultado: 45.92 46 50
- SQL> SELECT TRUNC(45.923, 2), TRUNC(45,923), TRUNC(45.923, -1)
FROM SYS.DUAL;
- Resultado: 45.92 45 40

S.Q.L.

Trabajando con fechas

- Oracle almacena fechas en un formato numérico interno de 7 bytes:
 - Siglo, año, mes, día, horas, minutos, segundos
- El formato de fecha por defecto es DD-MON-YY
- SYSDATE es una función que devuelve fecha y hora (pseudocolumna del sistema)
- DUAL es una tabla virtual de la bd., que puede ser usada para inspeccionar SYSDATE.

S.Q.L.

Operadores aritméticos de fechas

- Sumar o restar un número a/o de una fecha da por resultado una fecha.
- Restar dos fechas para encontrar la cantidad de días entre esas fechas.
- Sumar horas a una fecha dividiendo la cantidad de horas por 24.

S.Q.L.

Funciones de Fecha (I)

- MONTHS_BETWEEN (fecha1, fecha2)
 - Número de meses entre dos fechas. El resultado puede ser positivo o negativo.
- ADD_MONTHS (fecha, n)

- Añade n meses a fecha, según calendario. N debe de ser un número entero y puede ser negativo.
- NEXT_DAY (fecha, 'carácter')
 - Devuelve la fecha del día especificado ('carácter') siguiente a fecha. Carácter puede ser un número representando un día o una cadena de caracteres, p.ej. 'FRIDAY'.

S.Q.L.

Funciones de Fecha (II)

- LAST_DAY (fecha)
 - Devuelve la fecha del último día del mes que contiene fecha.
- ROUND (fecha [, 'fmt'])
 - Cuando no se especifica ningún formato, devuelve la fecha del primer día del mes contenido en fecha. Si fmt=YEAR, encuentra el primer día del año.
- TRUNC (fecha [, 'fmt'])
 - Devuelve la fecha con la porción del día truncado en la unidad especificada por el modelo de formato fmt. Si se omite el formato, la fecha se trunca en el día más próximo.

S.Q.L.

Ejemplos funciones de fecha

- MONTS_BETWEEN ('01-SEP-95', '11-JAN-94') → 19.6774194
- ADD_MONTHS('11-JAN-94', 6) → '11-JUL-94'
- NEXT_DAY ('01-SEP-95', 'FRIDAY') → '08-SEP-95'
- LAST_DAY ('01-SEP-95') → '30-SEP-95'
- ROUND ('25-JUL-95', 'MONTH') → '01-AUG-95'
- ROUND ('25-JUL-95', 'YEAR') → '01-JAN-96'
- TRUNC ('25-JUL-95', 'MONTH') → '01-JUL-95'

- TRUNC ('25-JUL-95', 'YEAR') ➡ '01-JAN-95'

S.Q.L.

Formatos de Fecha (I)

- YYYY / YEAR
 - Año completo en número / Año en letras
- MM / MONTH
 - Nº del mes con dos dígitos / Nombre completo del mes
- DY / DAY
 - Día de la semana en tres letras / Nombre completo del día
- fm (fill mode)
 - Elimina los espacios en blanco de relleno o suprime ceros a la izquierda

S.Q.L.

Formatos de Fecha (II)

- Obtención de la hora:
 - HH / HH12 / HH24
 - Hora del día / Hora (1-12) / Hora (1-24)
 - MI / SS / SSSS
 - Minutos / Segundos / Segundos después de medianoche
 - AM o PM
 - Indicador del Meridiano
 - Sufijo SP / SPTH o THSP
 - Deletreo del número / Deletreo números ordinales
- Se permiten literales

S.Q.L.

Funciones de conversión (I)

- La conversión de tipos de datos puede ser:
 - IMPLÍCITA: Realizada automáticamente por Oracle
 - EXPLÍCITA: El usuario es quien la realiza
- Conversión Implícita de datos
 - De VARCHAR2 o CHAR → a NUMBER
 - De VARCHAR2 o CHAR → a DATE
 - De NUMBER → a VARCHAR2
 - De DATE → a VARCHAR2
- Estas conversiones se realizan por asignaciones, si Oracle 8 puede convertir el tipo de dato del valor utilizado en la asignación en el tipo de dato que era el objetivo de la asignación.

S.Q.L.

Funciones de conversión (II)

- TO_CHAR (número | fecha [, 'fmt'])
 - Convierte un número o fecha en una cadena de caracteres VARCHAR2 con el modelo de formato fmt.
 - 9: Representa un número
 - 0: Fuerza a que se muestra el cero
 - \$: Signo de dólar
 - L: Usa el signo de moneda local
 - .: Imprime el punto decimal
 - :: Imprime el indicador de millar
 - Para fechas, los fmt anteriores.

S.Q.L.

Funciones de conversión (III)

- **TO_NUMBER (char)**
 - Convierte una cadena de caracteres con dígitos en un número.
- **TO_DATE (char [, 'fmt'])**
 - Convierte una cadena de caracteres representando una fecha en un valor de fecha según el fmt especificado. Si se omite el fmt, el formato es DD-MON-YY.
- **NVL (expr1, expr2)**
 - Convierte un nulo (expr1) a un valor de tipo fecha, cadena o número (expr2).

S.Q.L.

La Función DECODE

- Hace las veces de sentencia CASE o IF-THEN-ELSE, para facilitar consultas condicionales.
- *Descifra* una expresión después de compararla con cada valor de búsqueda. Si la expresión es la misma que la búsqueda, se devuelve el resultado. Si se omite el valor por defecto, se devolverá un valor nulo donde una búsqueda no coincida con ninguno de los valores resultantes.

S.Q.L.

Uso de DECODE

- SQL>

SELECT	job,	sal,
DECODE (job, 'ANALYST', sal*1.1, 'CLERK', sal*1.15, 'MANAGER', sal*1.20, sal) AS		
"Nuevo		salario"
FROM emp;		
- *Si job = 'ANALYST' entonces el salario se incrementa en un 10%*
- *Si job = 'CLERK' entonces se incrementa en un 15%*
- *Si jog = 'MANAGER' entonces se incrementa en un 20%*
- *Para otro caso, entones no hay incremento de salario*

TEMA 4

FUNCIONES DE GRUPO

S.Q.L.

Funciones de Grupo (I)

- AVG ([DISTINCT | ALL] n)
 - Valor promedio de n.
- COUNT ({* | [DISTINCT | ALL] expr})
 - Cantidad de filas con expr no nulo. Con * se cuentan todas las filas incluyendo duplicadas y valores nulos.
- MAX ([DISTINCT | ALL] expr)
 - Valor máximo de expr.
- MIN ([DISTINCT | ALL] expr)
 - Valor mínimo de expr., ignorando los valores nulos.

S.Q.L.

Funciones de Grupo (II)

- STDDEV ([DISTINCT | ALL] n)
 - Desviación estándar de n, ignorando los valores nulos.
- SUM ([DISTINCT | ALL] n)
 - Suma los valores de n, ignorando los valores nulos.
- VARIANCE ([DISTINCT | ALL] n)
 - Varianza de n, ignorando los valores nulos
- *Estas funciones no se pueden usar en la cláusula WHERE.*

S.Q.L.

Funciones de grupo y Nulos

- Las funciones de grupo IGNORAN los valores nulos de las columnas.
- ¿Qué resultado obtendríamos si calculamos la media de la comisión de los empleados?

S.Q.L.

NVL y funciones de grupo

- Esta media no es correcta porque se han ignorado las filas cuya comisión es nula.
- Solución: Uso de la función NVL para forzar a las funciones de grupo que admitan los valores nulos.

S.Q.L.

Sintaxis de GROUP BY

- Crea grupo de datos, por lo tanto se pueden usar funciones de grupo para devolver información resumida para cada grupo.

S.Q.L.

Uso de GROUP BY (I)

- Si se incluye una función de grupo en una cláusula SELECT, no se puede seleccionar resultados individuales a menos que la columna aparezca en la cláusula GROUP BY.
- No se pueden usar alias en GROUP BY.
- Por defecto, tras un GROUP BY, las filas se ordenan de forma ascendente
- Ejemplo:
>SQL> SELECT deptno, AVG(sal) FROM emp GROUP BY deptno;

S.Q.L.

Uso de GROUP BY (II)

- La columna referenciada por GROUP BY no es necesario seleccionarla.
- Todas las columnas mencionadas en la SELECT que no son funciones de grupo, tienen que estar en la cláusula GROUP BY.
- Se pueden formar agrupaciones sobre múltiples columnas:
`>SQL> SELECT deptno, job, sum(sal) FROM emp GROUP BY deptno, job;`

S.Q.L.

Consultas no válidas

- Cualquier columna o expresión en la SELECT que no sea una función agregada, tiene que ser especificada en la cláusula GROUP BY
`SQL> SELECT deptno, COUNT(ename) FROM emp;`
- No puede usar una cláusula WHERE para restringir grupos. Utilice la cláusula HAVING para restringir grupos.
`SQL> SELECT deptno, AVG(sal) FROM emp WHERE AVG(sal) > 2000 GROUP BY deptno;`

S.Q.L.

Cláusula HAVING

- Use la cláusula HAVING para restringir grupos:
 - Los registros son agrupados
 - Se aplica la función de grupo
 - Los grupos que se corresponden con la cláusula HAVING se visualizan (condición TRUE).
- HAVING puede preceder a GROUP BY, pero se recomienda que se ponga en primer lugar GROUP BY porque es más lógico. *(1ª se calculan grupos y posteriormente se calcula HAVING sobre esos gpos.).*

S.Q.L.

Sintaxis de HAVING

TEMA 5

JOIN

S.Q.L.

Concepto de JOIN

- Un JOIN se utiliza para consultar datos de más de una tabla
- La condición de JOIN se escribe en la cláusula WHERE.
- Si existen columnas con el mismo nombre en las tablas seleccionadas, se deberán nombrar los campos
- Ejemplo:

S.Q.L.

Tipos de JOIN

- Existen dos tipos principales de JOIN:
 - EQUIJOIN → Join sobre dos o más tablas, por igualdad de campos.
 - NON-EQUIJOIN → Por desigualdad, sin correspondencia directa entre campos de tablas. La relación se puede establecer mediante criterios de rango (<, >, BETWEEN, ...)
- Y dos más adicionales:
 - OUTER JOIN → Para ver, también, las filas que no complen la condición de Join. El operador de un Outer Join es el signo más (+), en el "lado" del join que es deficiente en información.
 - SELF JOIN → Combinación de una tabla consigo misma.

S.Q.L.

Ejemplo de Equijoin

- Ya que la columna DEPTNO es igual en ambas tablas, ésta debe ir prefijada por el nombre de la tabla para evitar la ambigüedad.

S.Q.L.

Ejemplo de Non-Equijoins

- En este ejemplo se han usado alias de tablas (e para la tabla emp y s para la tabla salgrade).

S.Q.L.

Ejemplo de Outer Join

- En este ejemplo se muestran los números y nombres de todos los departamentos, incluidos aquellos que no tienen empleado.
- Si se le añade: **AND emp.deptno is null**, sólo se mostrarían las no coincidencias.

S.Q.L.

Ejemplo de Self Join

- En este ejemplo la cláusula WHERE contiene la combinación "*dónde un jefe de un trabajador coincide con el número de empleado para el jefe*".

TEMA 6

SUBCONSULTAS

S.Q.L.

¿Cuándo subconsultas?

- ¿Quién tiene un salario superior al de Jones

S.Q.L.

Sintaxis de una subconsulta

- La subconsulta se ejecuta una vez y antes de la consulta principal.
- El resultado de ella es usado por la consulta principal externa.

S.Q.L.

Guía Uso de Subconsultas

- Encierre las subconsultas entre paréntesis.

- No añada una cláusula ORDER BY a una subconsulta.
- Utilice operadores a nivel de fila para subconsultas que devuelvan solo una fila → MONOREGISTRO.
- Utilice operadores que actúan sobre varios registros para subconsultas que devuelven más de una fila → MULTIREGISTRO.

S.Q.L.

Subconsultas Mono-registro

- Devuelven un único registro.
- Se utilizan operadores de comparación (=, >, >=, <, <= y <>).
- Ejemplo:

S.Q.L.

Subconsultas Multi-registro

- Devuelven más de un registro
- Se utilizan comparadores multiregistro:
 - IN → TRUE si se encuentra en la lista.
 - ANY (y sinónimo SOME) → TRUE si la condición se cumple con algún registro de la lista devuelta por la subconsulta.
 - ALL → TRUE si la condición se cumple con todos los registros de la lista devuelta por la subconsulta.
- El operador NOT puede ser utilizado con los operadores IN, ANY y ALL.

S.Q.L.

Ejemplo subc. Multi-registro

S.Q.L.

Subcons. en cláusula FROM

- Puede utilizar una subconsulta en una cláusula FROM de una sentencia SELECT:

- *Este ejemplo muestra los nombres, salarios, núm. Departamentos y media de salarios, de todos los empleados que cobran más que la media de salarios de su departamento.*

TEMA 7

D.M.L.

S.Q.L.

Manipulación de Datos (DML)

- Sentencias DML son:
 - INSERT → Añade registros a una tabla.
 - UPDATE → Modifica registros existentes de una tabla.
 - DELETE → Elimina registros existentes de una tabla.

S.Q.L.

Control de Datos (DCL)

- Estas sentencias se completan con los comandos de control de transacción (DCL), las cuales aseguran la consistencia de los datos.
 - COMMIT → Finaliza la transacción actual haciendo que todos los cambios pendientes pasen a ser permanentes.
 - ROLLBACK → Finaliza la transacción en curso descartando todos los cambios pendientes.
 - SAVEPOINT → Establece una "marca" dentro de la transacción en curso, usada por COMMIT o ROLLBACK.

S.Q.L.

La Sentencia INSERT

- Mediante esta sentencia sólo se inserta un registro cada vez.
- El nombre de las columnas es opcional. Si se omiten se deben colocar los valores en el orden que las columnas tienen en la tabla.

- Caracteres y fechas entre comillas simples.

S.Q.L.

Inserción de Valores Nulos

- Método Implícito: Omitir la columna en la lista:
- Método Explícito: Especificar NULL o el string vacío (''), para cadenas y fechas, en la lista de VALUES:

S.Q.L.

Inserción Valores Especiales

- SYSDATE registra la fecha y hora actual:
- USERID inserta el nombre del usuario actual

S.Q.L.

Inserción reg. de otra tabla

- Se escribe el comando INSERT con una subconsulta.
- No usar la cláusula VALUES.
- Deben coincidir el número de columnas de INSERT con el de la subconsulta

S.Q.L.

La Sentencia UPDATE

- Los registros a modificar se especifican por medio de la cláusula WHERE.

- Si se omite WHERE se modificarían todos los registros de la tabla.

S.Q.L.

Modificación con subconsultas

- P.ej.: *Modificar el oficio y departamento del empleado 7698, con los valores correspondientes actualmente al empleado 7499:*

S.Q.L.

La Sentencia DELETE

- Los registros a eliminar se especifican en la cláusula WHERE.
- Si se omite WHERE se borrarán todos los registros de la tabla.

S.Q.L.

Eliminación con subconsulta

- Utilice subconsultas en sentencias DELETE, para eliminar registros de una tabla, basados en valores de otra tabla:

TEMA 8

D.D.L.

S.Q.L.

Definición de Datos (DDL)

- Sentencias DDL son:
 - CREATE TABLE → Crea una tabla. Para ello el usuario debe de tener el privilegio CREATE TABLE.
 - ALTER TABLE → Permite modificar la estructura definida para una tabla.

- **DROP TABLE** Elimina una tabla (datos y estructura) y sus índices. No se puede hacer Rollback de esta sentencia.
- **RENAME** Cambia el nombre de una tabla, vista, secuencia o sinónimo.

S.Q.L.

Sentencia CREATE TABLE

- Necesario tener privilegio CREATE TABLE.
- Ha de especificar:
 - Nombre de tabla
 - Para las columnas: nombre, tipo de dato y tamaño.

S.Q.L.

Reglas para los nombres

- Deben de comenzar con una letra.
- Pueden tener una longitud de 1 – 30 caracteres de largo.
- Deben contener solamente A-Z, a-z, 0-9, _, \$ y #.
- No deben duplicar el nombre de otro objeto que sea propiedad del mismo usuario o schema.
- No debe ser una palabra reservada del servidor Oracle8.

S.Q.L.

Tipos de Datos

VARCHAR2(tamaño)	Dato carácter de longitud variable. Máx. 4000.
CHAR(tamaño)	Dato carácter de longitud fija. Máx. 255.
NUMBER(p,s)	Dato numérico de longitud variable.p entre 1..38; s entre 84..127
DATE	Valores de fecha y hora. Entre el 1 Enero 4712 A.C. Y el 31 Diciembre del 4712 D.C.
LONG	Dato carácter de long.variable hasta 2 Gb.
CLOB	Dato carácter “single-byte” de hasta 4 Gb.

RAW(tamaño) y LONG RAW	Datos Binarios según tamaño especificado y Datos Binarios de long.variable hasta 2 Gb.
BLOB	Datos Binarios hasta 4 Gb.
BFILE	Datos binarios almacenados en fich. Externo. Hasta 4 Gb.

S.Q.L.

Creación de tabla por subconsulta

- Se puede crear una tabla e insertar filas combinando el comando CREATE TABLE con la opción AS subconsulta.
- Es necesario hacer coincidir la cantidad de columnas especificadas con las de la subconsulta.
- Si no se indican nombres de columnas, éstas serán los mismos que los de la subconsulta.

S.Q.L.

Creación de tabla por subconsulta

- Ejemplo:

S.Q.L.

Sentencia ALTER TABLE

- Permite añadir nuevas columnas o modificar una ya existente.

S.Q.L.

Añadir una Columna

- La nueva columna aparecerá en el último lugar de la tabla. No se puede especificar el orden.
- Puede añadir o modificar columnas, pero no eliminarlas de una tabla.
- Si la tabla ya contiene registros al añadir una nueva columna, ésta se inicializará

con valores nulos para todos los registros.

- Puede definir una columna NOT NULL sólo si la tabla está vacía.

S.Q.L.

Modificar una Columna

- Puede cambiar el tipo de datos de una columna, su tamaño y valor por defecto
- Si cambia el valor por defecto, afectará sólo a sucesivas inserciones en la tabla.

S.Q.L.

Sentencia DROP TABLE

- Se borra estructura, datos e índices de la tabla. Borrado Físico.
- No se puede hacer Rollback de la sentencia.
- Sólo el propietario de la tabla u otro usuario con el permiso DROP ANY TABLE puede eliminar una tabla.

S.Q.L.

Sentencia RENAME

- Permite cambiar el nombre de una tabla, vista, secuencia o sinónimo.
- Debe ser el propietario del objeto.

TEMA 9 VISTAS

S.Q.L.

Concepto de Vista

- Una vista es una tabla lógica basada en una tabla u otra vista.

- No contiene datos en sí misma, pero es como una ventana a través de la cual se pueden ver o cambiar los datos de las tablas.
- Podemos representar con ellas subconjuntos lógicos o combinaciones de datos.
- Las tablas sobre las cuales se basa una vista se llaman tablas base.
- Se almacenan en el Diccionario de Datos, USER_VIEWS.

S.Q.L.

¿Por qué usar Vistas?

- Para restringir el acceso a la B.D.
- Para realizar consultas complejas de manera fácil.
- Para obtener una independencia de los datos
- Para presentar diferentes vistas de los mismos datos.

S.Q.L.

Creación de una Vista

- **FORCE:** Crea la vista sin importar que la tabla base exista o no.
- **WITH CHECK OPTION:** Especifica que solamente las filas accesibles a la vista pueden ser insertadas o actualizadas.
- **CONSTRAINT:** Nombre asignado a la restricción CHECK OPTION.
- **WITH READ ONLY:** Asegura que ninguna operación DML pueda realizarse sobre esta vista.

S.Q.L.

Ejemplo creación de Vista

S.Q.L.

Eliminación de una Vista

- Al borrar una vista no perderá los datos, porque la vista está basada en tablas subyacentes de la B.D.
- Únicamente el creador o un usuario con el privilegio DROP ANY VIEW puede eliminar una vista.

S.Q.L.

Limitaciones DML en Vistas

- Se pueden realizar operaciones DML sobre vistas simples.
- No se puede eliminar una fila si la vista contiene Funciones de grupo, una cláusula GROUP BY o el comando DISTINCT.
- No es posible modificar datos en la vista si contiene cualquiera de las condiciones anteriores, columnas definidas por expresiones o la pseudocolumna ROWNUM
- No se puede agregar datos si la vista contiene cualquiera de las condiciones anteriores o cualquier columna NOT NULL no incluida por la vista (tabla base).

S.Q.L.

Creación de Sinónimos

- Simplifican el acceso a los objetos al crear otro nombre para un objeto (sinónimo).
- Hacen referencia a una tabla propia o de otro usuario.
- Permite acortar la longitud de los nombre de los objetos a la vez que elimina la necesidad de cualificar el objeto con un esquema.
- El DBA puede crear un sinónimo público accesible a todos los usuarios.

S.Q.L.

Eliminación de Sinónimos

- Sólo el DBA puede eliminar un sinónimo público.

TEMA 10

ENTORNO SQL*PLUS

SQL*PLUS

Entorno de ORACLE

- SQL*PLUS es una herramienta de Oracle que reconoce y envía sentencias SQL al servidor Oracle para su ejecución.
- Contiene su propio lenguaje de comandos.
- Permite abreviatura de palabras claves de SQL*PLUS.
- Permite guardar y recuperar sentencias SQL en archivos.

SQL*PLUS

Conexión a SQL*PLUS

- Desde un entorno Windows:
- Desde la línea de comandos: `sqlplus [username[/password[@database]]]`

SQL*PLUS

Comandos de Edición

- A[PPEND] texto
- C[HANGE] /antiguo /nuevo
- C[HANGE] /texto /
- CL[EAR] BUFF[ER]
- DEL [n] [m]
- I[INPUT] [texto]
- L[IST] [n] [m]

- R[UN]
- n [texto]
- 0 texto

SQL*PLUS

Comandos de Ficheros

- SAV[E] nombre_fichero[.ext] [REP[LACE] | APP[END]]
- GET nombre_fichero[.ext]
- STA[RT] nombre_fichero[.ext]
- @nombre_fichero[.ext]
- ED[IT] nombre_fichero[.ext]
- SPO[OL] [nombre_fichero[.ext] | OFF | OUT]
- EXIT

SQL*PLUS

Variables de Sustitución

- Usar variables de sustitución de SQL*PLUS para almacenar valores en forma temporal con **&variable**.
- El doble ampersand **&&** para cuando se quiere reutilizar el valor de una variable sin tener que preguntarle al usuario cada vez.

SQL*PLUS

Uso de variables sustitución

- Use una variable precedida con un ampersand (&) para pedir un valor al usuario:
- Si la variable no existe, SQL*PLUS le solicita al usuario un valor. SQL*PLUS no descarta la nueva variable una vez que esta es usada.

SQL*PLUS

Caracteres y Fechas con Var. Sust.

- Use comillas simples para fechas y cadenas de caracteres:
- También pueden usarse funciones como UPPER y LOWER con el ampersand:

SQL*PLUS

Comando SET VERIFY

- SET VERIFY { ON | OFF }
- Definiendo este parámetro en ON, fuerza a SQL*PLUS a visualizar el texto de un comando antes y después de que la variable sea remplazada con valores:

SQL*PLUS

Variables de Usuario

- Se predefinen variables usando uno de los dos comandos SQL*PLUS siguientes:
 - DEFINE → Crea una variable de usuario de tipo CHAR.
 - ACCEPT → Acepta un valor introducido por el usuario y lo almacena en una variable, además de permitir introducir literales informativos así como definir el tipo de variable.

SQL*PLUS

Comando DEFINE/UNDEFINE

- Ejemplo de definición de variable:
- Ejemplo de uso de variable en SQL:

- UNDEFINE elimina la definición de una variable.

SQL*PLUS

Comando ACCEPT

- Crea un *prompt* personalizado para aceptar datos del usuario.
- Define explícitamente una variable de tipo NUMBER o DATE.
- Le oculta al usuario el dato que esté introduciendo, por razones de seguridad.

SQL*PLUS

Personalización del Entorno

- Use comandos SET para controlar la sesión actual.
- Verifique los valores que tiene definidos por medio del comando SHOW.

SQL*PLUS

Parámetros del Entorno

- ARRAY[SIZE] { 20 | n }
- COLSEP { _ | texto }
- FEED[BACK] { 6 | n | ON | OFF }
- HEA[DING] { OFF | ON }
- LIN[ESIZE] { 80 | n }
- LONG { 80 | n }
- PAGES[IZE] { 24 | n }
- PAU[SE] { OFF | ON | text }

- TERM[OUT] { OFF | ON }

SQL*PLUS

Comandos de Formato

- COL[UMN] [opción_de_columna]
- BRE[AK] [ON elemento_del_informe]
- TTI[TLE] [texto | OFF | ON]
- BTI[TLE] [texto | OFF | ON]

SQL*PLUS

Comando COLUMN

- Controla la visualización de una columna:
- CLE[AR] → Limpia cualquier formato definido.
- FOR[MAT] formato → Cambia la visualización de la columna según un modelo de formato.
- HEA[DING] texto → Define la cabecera para una columna
- JUS[TIFY] {alineación} → Alinea la cabecera de una columna a la izqda., centro o derecha.
- PRI[NT] / NOPRI[NT] → Muestra / Oculta la columna.
- NUL[L] texto → Especifica el texto a visualizar en caso de valores nulos.
- WRA[PPED] → Continúa escribiendo en las líneas siguientes.

SQL*PLUS

Formatos con COLUMN

Elemento	Descripción	Ejemplo	Resultado
An	Ancho de columna de n		
9	Suprime ceros	999999	1234
0	Fuerza ceros	099999	01234

\$	Signo dólar flotante	\$9999	\$1234
L	Moneda local	L9999	L1234
.	Posición del punto decimal	9999.99	1234.00
,	Separador de miles	9,999	1,234

SQL*PLUS

Comando BREAK

- Suprime duplicados y secciona registros:
 - Para suprimir duplicados:
SQL> BREAK ON ename ON job
 - Para producir totales globales:
SQL> BREAK ON report
 - Para seccionar valores según la ruptura:
SQL> BREAK ON ename SKIP 4 ON job SKIP2
 - Limpiar todas las definiciones BREAK:
SQL> CLEAR BREAK

SQL*PLUS

Comandos TTITLE y BTITLE

- Sintaxis de TTITLE y BTITLE:
- Definir cabecera del informe:
SQL> TTITLE 'Informe de|Salarios'
- Definir el pie de página del informe:
SQL> BTITLE 'Confidencial'

TEMA 11

PL/SQL

PL/SQL

¿Qué es PL/SQL?

- Lenguaje de procesamiento procedimental.
- Implementado por Oracle.
- Dispone de estructuras de programación similares a las de la mayoría de los lenguajes de programación.
- Objetivo: Interactuar con la B.D.

PL/SQL

Estructura Bloques Anónimos

- **[DECLARE]**
Define objetos PL/SQL que serán utilizados dentro del mismo bloque
- **BEGIN**
Sentencias Ejecutables
- **[EXCEPTION]**
Qué hacer si la acción ejecutada causa error
- **END;**

PL/SQL

Estructura de Procedimiento

- **PROCEDURE** nombre **IS**
Sección Declarativa
- **BEGIN**
Sección Ejecutable
- **[EXCEPTION]**
Sección de Excepciones
- **END;**

PL/SQL

Estructura de Función

- **FUNCTION** nombre **RETURN** tipo_dato **IS**
Sección Declarativa

- **BEGIN**
Sección Ejecutable
- **[EXCEPTION]**
Sección de Excepciones
- **END;**

PL/SQL

Declaración de Variables

- Reglas para nombres:
 - Dos variables pueden tener el mismo nombre, si están en bloques diferentes.
 - El nombre de la variable (identificador) no debería ser el mismo que el de una columna de una tabla utilizada en el bloque.

PL/SQL

Asignación e Inicialización de Variables

- Asignación:
- Inicialización:
- Por defecto, todas las variables se inicializan a NULL.

PL/SQL

Tipos de Variables

- ESCALARES
- BOOLEANAS
- COMPUESTAS

- LOB
- DE ENLACE (BIND)

PL/SQL

Variables Escalares

- VARCHAR2 (longitud_máxima)
- NUMBER [(precisión, escala)]
- DATE
- CHAR [(longitud_máxima)]
- LONG
- LONG RAW
- BOOLEAN
- BINARY_INTEGER

PL/SQL

Atributo %TYPE

- Permite declarar una variable basada en:
 - Otras variables previamente declaradas
 - La definición de una columna de la base de datos
- Preceder de %TYPE por:
 - La tabla y la columna de la base de datos
 - El nombre de la variable definida con anterioridad
- Ejemplo:
v_ename
emp.ename%TYPE;

PL/SQL

Variables BOOLEANAS

- A una variable Boolean sólo se le pueden asignar los valores: **TRUE**, **FALSE** o **NULL**.
- Estas variables están conectadas por los operadores lógicos **AND**, **OR** y **NOT**.

PL/SQL

Tipos de Datos Compuestos

- Tipos:
 - REGISTROS PL/SQL
 - TABLAS PL/SQL
- Contienen componentes internos

PL/SQL

Creación de un Registro

- Sintaxis:
- Donde declaración_campo significa:

PL/SQL

Ejemplo de Registro PL/SQL

- Declarar un registro para almacenar el número de empleado, nombre, trabajo y sueldo de un nuevo empleado:
- Los componentes individuales del registro se referenciarán de forma cualificada; en este ejemplo:

PL/SQL

El Atributo %ROWTYPE

- Define un registro con la estructura de la tabla o vista de la B.D.
- Los campos del registro toman sus nombres y tipos de datos de las columnas de la vista o tabla.
- Ejemplo:
DECLARE registro_empleado emp%ROWTYPE;

PL/SQL

Ventajas de %ROWTYPE

- El número y los tipos de datos de las columnas de la tabla pueden no ser conocidos.
- Simplifica la programación al no tener que definir explícitamente los campos y tipos del registro.
- Es útil para realizar recuperaciones de filas con la sentencia SELECT:

PL/SQL

Tablas PL/SQL

- Cuentan con dos componentes:
 - TIPO DE DATOS DE CLAVE PRIMARIA BINARY_INTEGER
 - COLUMNA DE TIPO DE DATOS ESCALARES O DE REGISTRO.
- Aumentan dinámicamente porque no tienen restricciones.
- Se almacenan en memoria.

PL/SQL

Creación de Tablas PL/SQL

- Sintaxis:

- Ejemplo:

PL/SQL

Creación de Tablas PL/SQL

PL/SQL

Variables LOB

- Permiten almacenar bloques de datos no estructurados, como pantallas de texto, imágenes gráficas, vídeo clips, y sonido, de hasta 4 Gb. de tamaño.

CLOB *Character Large Object*. Se utiliza para almacenar bloques grandes de datos de caracteres.

BLOB *Binary Large Object*. Se utiliza para almacenar objetos binarios grandes en la B.D.

BFILE *Binary File*. Se utiliza para almacenar objetos binarios grandes en archivos del sistema operativo, fuera de la B.D.

NCLOB *National Language Character Large Object*. Se utiliza para almacenar en la B.D. bloques grandes de datos NCHAR de un byte único o multi-bytes de ancho fijo.

PL/SQL

Variables de Enlace (Bind)

- Son variables de SQL*Plus.
- Las pueden referenciar bloques de PL/SQL mediante el uso del ampersand (&).
- Ejemplo:

S.Q.L.

Ejercicios



- Basados en las tablas emp y dept del usuario Scott de la database Oracle.
- Crear el modelo de datos necesario para representar la actividad de un Videoclub.
- Mostrar los nombres de los empleados del deptno 10, junto con los meses que llevan trabajando en la empresa.
- Extraer el nombre de empleado y su fecha de alta en la empresa con formato Dia (letras) , mes (letras) , año, horas, minutos y segundos.
- Mostrar el nº de empleado, job, nombre, así como la comisión. Si no tiene comisión, poner el texto 'Sin Comisión'. La información debe de salir ordenada por nº de empleado en descendente.
- Nº de empleado con el salario mas alto.
- Nombre de deptno y Nombre de empleado que mas tarde entró en la empresa, para cada uno de los departamentos.
- Incrementar un 20% el salario de todos los empleados que no son de Bostón.
- Borrar los empleados cuyo nombre no comienza por A , que no son Analistas y que trabajan en New York.
- Nombre de empleado, suma de salario y comisión para los empleados cuyo salario es superior a la media de su departamento.

TEMA 12

GENERALIDADES DE PL/SQL

PL/SQL

Generalidades de PL/SQL (I)

- Comentarios en PL/SQL:
 - Anteponga a los comentarios de una sola línea dos guiones (--) a modo de prefijo.

- Coloque comentarios de varias líneas entre /* y */.

- Operadores de PL/SQL:

** , NOT	Exponenciación, negación lógica
+, -	Identidad, negación
*, /	Multipliación, división
+, -,	Suma, resta, concatenación
=, !=, <, >, <=, >=, IS NULL, LIKE, BETWEEN, IN	Comparación
AND	Conjunción
OR	Inclusión

PL/SQL

Generalidades de PL/SQL (II)

- Bloques Anidados y Ámbito de la Variable:

PL/SQL

Generalidades de PL/SQL (III)

- Para hacer referencia a una variable de sustitución en PL/SQL debe anteponer a su nombre dos puntos (:) a modo de prefijo:

PL/SQL

Generalidades de PL/SQL (IV)

- Directrices de Programación para facilitar el mantenimiento del código:
 - Documente el código con comentarios.
 - Desarrolle una convención de mayúsculas/minúsculas para el código.
 - Desarrolle convenciones de nomenclatura para identificadores y otros objetos.
 - Sangre el código para facilitar la lectura.
 - Evite la ambigüedad entre variables locales, parámetros formales y nombres de columnas de las tablas de la B.D.

PL/SQL

Generalidades de PL/SQL (V)

- Cuando trabaje con valores nulos puede evitar algunos de los errores más habituales si recuerda las siguientes reglas:
 - Las comparaciones en las que se incluyen valores nulos siempre resultan NULL.
 - Si se aplica el operador lógico NOT a un valor nulo resulta NULL.
 - En las sentencias de control condicionales, si la condición resulta NULL, no se ejecutarán las sentencias asociadas.

PL/SQL

Sentencias SQL en PL/SQL

- SELECT recupera exactamente UNA fila.
- INSERT añade una fila.
- UPDATE modifica una o más filas existentes.
- DELETE suprime una o más filas existentes.
- COMMIT hace permanentes todas las modificaciones pendientes.
- ROLLBACK elimina todas las modificaciones pendientes.
- SAVEPOINT marca un punto intermedio en el procesamiento de las transacciones.

PL/SQL

Sentencia SELECT

- Sintaxis:
 - Recuerde, sólo se debe de recuperar una fila. Más de una fila provocará errores.

PL/SQL

Ejemplo de SELECT

- Recupere la suma de los salarios de todos los empleados de un departamento específico:

PL/SQL

Inserción de Datos

- Añada nueva información sobre los empleados en la tabla emp:

PL/SQL

Actualización de Datos

- Aumente el salario de todos los empleados de la tabla emp que son Analistas:

PL/SQL

Supresión de Datos

- Suprima filas pertenecientes al departamento 10 de la tabla emp:

PL/SQL

Control de Transacciones

- COMMIT finaliza la transacción actual realizando todos los cambios pendientes en la B.D.
- ROLLBACK finaliza la transacción actual desechando todos los cambios pendientes.

PL/SQL

Control de Transacciones

PL/SQL

Control Flujo de Ejecución

- Puede modificar el flujo lógico de sentencias utilizando sentencias IF condicionales y estructuras de control de bucles.
- Sentencias IF condicionales:
 - IF-THEN
 - IF-THEN-ELSE
 - IF-THEN-ELSIF
- Control de bucles:
 - Bucle básico LOOP
 - Bucle FOR
 - Bucle WHILE

PL/SQL

Sentencia IF

- Sintaxis:

PL/SQL

Flujo de IF-THEN-ELSE

PL/SQL

Flujo de IF-THEN-ELSIF

PL/SQL

Condiciones Booleanas

PL/SQL

Bucle Básico: LOOP

- Sintaxis:
- Donde condición es una expresión o variable booleana (TRUE, FALSE o NULL).

PL/SQL

Bucle FOR

- Sintaxis:
- No declare el índice; se declara implícitamente como un BINARY_INTEGER. Fuera del bucle el índice no está definido.
- Los límites desde..hasta deben de ser literales numéricos. Pueden ser expresiones que se convierten en valores numéricos.

PL/SQL

Bucle WHILE

- Sintaxis:
- La condición se evalúa al inicio de cada iteración

PL/SQL

Etiquetas y Loops Anidados

- Puede anidar bucles a varios niveles.
- Utilice etiquetas para distinguir entre los bloques y los bucles.

- Salga al bucle externo con la sentencia EXIT que hace referencia a la etiqueta.
- Los nombres de etiquetas deben ir antes de la palabra LOOP y entre los delimitadores << >>.

PL/SQL

Etiquetas y Loops Anidados

- Ejemplo:

S.Q.L.

Ejercicios Bloques Anónimos



- Basados en las tablas Emp & Dept del esquema del usuario Scott de Oracle
- Actualizar los vendedores con una comisión mayor que 350\$ con un incremento del 15% de su salario. Si la operación afecta a mas de tres empleados, deshacer la transacción, en cualquier otro caso validar la transacción. Informar por pantalla de la operación realizada.
- Insertar en la tabla TEMP 100 filas. En la primera columna se insertará un índice secuencial (1, 2, 3...) y en la segunda columna un comentario indicando si el número generado es par o impar.
- Actualizar el trabajo a DIRECTOR a todos aquellos empleados cuyo salario sea mayor que 2000\$. Almacenar el número de empleados actualizados por la operación en la tabla TEMP. Si los afectados son mas de cinco personas, borrar los empleados cuyo salario sea mayor que 3000\$, insertar en la tabla TEMP el número de empleados borrados y validar la transacción.
- Calcular por medio de un bloque PL/SQL 9i anónimo el total de ganancias de los empleados (salario y comisión) y cuántos de éstos tienen un salario superior a 2000\$ para todos los departamentos de la empresa.

TEMA 13

CURSORES

PL/SQL

Definición de Cursor

- Útiles para las consultas que devuelven más de una fila.
- Son declarados y nombrados por el programador, y manipulados por medio de sentencias específicas en las acciones ejecutables del bloque.

PL/SQL

Control de Cursores

- 1º. Crear un área SQL específica → **DECLARE**
- 2º. Identificar el juego activo → **OPEN**
- 3º. Cargar la fila actual en variables → **FETCH**
- 4º. Si todavía existen filas sin leer, volver a 3º.
- 5º. Si no existen más filas a leer → **CLOSE**

PL/SQL

Declaración del Cursor

- Sintaxis:
- No incluya la cláusula INTO en la declaración del cursor.
- Si es necesario procesar filas en algún orden, incluya la cláusula ORDER BY.

PL/SQL

Ejemplo de Declaración

PL/SQL

Apertura del Cursor

- Sintaxis:
- Si la consulta no devuelve ninguna fila, no se producirá ninguna excepción al abrir el cursor.
- Para los cursores declarados con la cláusula FOR UPDATE, la sentencia OPEN bloquea estas filas.

PL/SQL

Recuperación de Datos

- Sintaxis:
- Incluya, en el FETCH, el mismo número de variables que las definidas en el cursor, y en el mismo orden.
- Compruebe si el cursor tiene filas.

PL/SQL

Cierre del Cursor

- Sintaxis:
- Cierre el cursor una vez completado el procesamiento de las filas.
- Si es necesario, vuelva a abrir el cursor.
- No intente recuperar los datos de un cursor una vez que ha sido cerrado (INVALID_CURSOR).

PL/SQL

Atributos de Cursores

- Proporcionan información de estado del cursor:

PL/SQL

El Atributo %ISOPEN

- Utilice el atributo de cursor %ISOPEN antes de ejecutar una recuperación para comprobar si el cursor está abierto.
- Ejemplo:

PL/SQL

Atributos %NOTFOUND, %ROWCOUNT

- Utilice %ROWCOUNT para recuperar un número exacto de filas.
- Utilice %NOTFOUND para determinar cuándo salir del bucle de lectura del cursor.
- Antes de la primera recuperación, %NOTFOUND es NULL, así que si FETCH no se ejecuta nunca satisfactoriamente, no se saldría nunca del bucle de lectura.

PL/SQL

Ejemplo %NOTFOUND, %ROWCOUNT

- Ejemplo:

PL/SQL

Bucles FOR de Cursor

- Sintaxis:
- Apertura, recuperación y cierre implícitos.
- No declare el registro, está declarado implícitamente.

PL/SQL

Cursores con Parámetros

- Sintaxis:
- Permite abrir un cursor varias veces con un juego activo distinto cada vez.

PL/SQL

Cursores con Parámetros

- Cada parámetro formal de la declaración del cursor debe tener un parámetro real correspondiente en la sentencia OPEN.
- La sintaxis de los parámetros es:

PL/SQL

Cursores con Parámetros

- Transfiera el número de departamento y el cargo a la cláusula WHERE:

PL/SQL

Cláusula FOR UPDATE

- Sintaxis:
- El bloqueo explícito le permite denegar el acceso mientras dura una transacción.
- Bloquee las filas antes de la actualización o supresión.
- La cláusula FOR UPDATE es la última cláusula de una sentencia SELECT, incluso después del ORDER BY.
- NOWAIT devuelve un error de Oracle si las filas han sido bloqueadas por otra sesión, de lo contrario se espera.

PL/SQL

Cláusula WHERE CURRENT OF

- Sintaxis:
- Incluya la cláusula FOR UPDATE en la definición del cursor para bloquear las filas.
- Especifique WHERE CURRENT OF en la sentencia UPDATE o DELETE para referirse a la fila actual del cursor.

S.Q.L.

Ejercicios Cursores



- Incrementar la comisión, en función del salario, de los empleados de Bostón y Nueva York según su antigüedad y cargo, según la siguiente tabla:

TEMA 14 EXCEPCIONES

PL/SQL

Concepto de Excepción

- ¿Qué es una excepción? Es un identificador de PL/SQL que surge durante la ejecución.
- ¿Cómo surge? Se produce por un error Oracle o bien puede ser provocada explícitamente.
- ¿Cómo se gestiona? Interrumpiéndola con un manejador de excepciones o propagándola al entorno de llamadas.

PL/SQL

Interrupción de Excepciones

- Sintaxis:

PL/SQL

Excepciones de Oracle8

PL/SQL

Excepciones de Oracle8

PL/SQL

Excepciones de Usuario

- Se declaran en la sección declarativa DECLARE.
- Se provocan explícitamente en la sección ejecutable utilizando la sentencia **RAISE**.
- Se gestiona la excepción dentro del bloque de excepciones EXCEPTION.

PL/SQL

Excepciones de Usuario

- Ejemplo:

PL/SQL

Funciones para Identificar Excepciones

- **SQLCODE** → Devuelve el valor numérico del código de error SQL. No se puede referenciar directamente, hay que asignarlo a una variable PL/SQL de tipo NUMBER.

- **SQLERRM** → Devuelve el mensaje asociado con el número de error. Tipo VARCHAR2.

PL/SQL

RAISE_APPLICATION_ERROR

- Sintaxis:
- Utilice el procedimiento RAISE_APPLICATION_ERROR para comunicar de forma interactiva una excepción predefinida, devolviendo un código y un mensaje de error no estándar.

PL/SQL

RAISE_APPLICATION_ERROR

- Se utiliza en dos lugares distintos:
 - SECCIÓN EJECUTABLE
 - SECCIÓN DE EXCEPCIONES
- Ejemplos:

SQL

Ejercicios

Ejercicios Excepciones



- Para un número de empleado dado (por medio de comandos SQL*Plus), incrementar el salario en un 10%. Controlar dos excepciones predefinidas de ORACLE, NO_DATA_FOUND y TOO_MANY_ROWS.
- Por medio de comandos SQL*Plus, aceptar tres valores que correspondan al número, nombre y localidad de un departamento. Realizar un bloque PL/SQL 9i que inserte en la tabla DEPT los valores aceptados.

Controlar los siguientes errores:

- Si el departamento ya existe, insertar en la tabla TEMP un error.
 - Si algún dato de los insertados es de mayor longitud que la especificada en la tabla, insertar un error en la tabla TEMP.
 - Si se producen otros errores, insertar en la tabla TEMP el número y el mensaje del error producido.
- Nota: El error ORACLE de longitud del dato fuera del rango es el -1438

TEMA 15

FUNCIONES PL/SQL

PL/SQL

Definición de Función

- Una función es un bloque nombrado PL/SQL que devuelve un valor.
- Una función puede estar almacenada en la B.D., como objeto de la B.D., para repetidas ejecuciones.
- Una función puede ser llamada como parte de una expresión.

PL/SQL

Sintaxis Creación Funciones

PL/SQL

Creación de una Función

PL/SQL

Desde dónde llamarlas

- Como columna de un SELECT.

- Condiciones en cláusulas WHERE y HAVING.
- Cláusulas ORDER BY y GROUP BY.
- Cláusula VALUES de un comando INSERT.
- Cláusula SET de un comando UPDATE.

PL/SQL

Restricciones en Funciones

- No se permiten comandos INSERT, UPDATE o DELETE.
- La función no puede llamar a otro subprograma que rompa una de las restricciones arriba indicadas.

PL/SQL

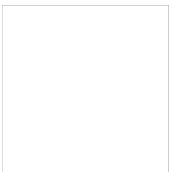
Borrado de Funciones

- Para quitar una función de la parte del servidor, utilizando SQL*Plus, ejecutar el comando DROP FUNCTION.
- Sintaxis:

SQL

Ejercicios

Ejercicios Funciones



- Crear la función ALTA_DEPT con tres argumentos: número, nombre y localidad del departamento. Este procedimiento debe comprobar que los números de departamento son múltiplos de 10, si no es así daría un error.
- Crear la función BAJA_DEPT con un argumento. Dado un número de departamento, borrarlo.
- Crear la función MOD_DEPT con dos argumentos. Dado un número de

departamento, actualizar la localidad al destino indicado en el parámetro. Informar con un booleano si se ha realizado correctamente la operación.